

AI Inventory Agent

An intelligent inventory management system with AI-powered forecasting, anomaly detection, and automated reorder suggestions. Built with Spring Boot, Flask, Bootstrap 5.3, and MySQL.

Tech Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5.3, Chart.js
Backend	Java 21, Spring Boot 3.2.5, Spring Data JPA, Sprin
AI Service	Python 3.14+, Flask, Pandas, NumPy, scikit-learn,
Database	MySQL 8.0 (XAMPP)
Build Tools	Maven 3.9.6, pip

Project Structure

```
AI-Inventory-Agent/
+-- backend/                # Spring Boot REST API
|   +-- pom.xml
|   +-- src/main/java/com/inventory/agent/
|       +-- InventoryAgentApplication.java
|       +-- config/         # SecurityConfig, CorsConfig, JwtU
|   til
|       +-- controller/     # REST controllers (8 controllers)
|       +-- dto/            # Request/Response DTOs
|       +-- entity/         # JPA entities (6 entities)
|       +-- exception/      # Global exception handler
|       +-- repo/           # Spring Data JPA repositories (6
|   repos)
|       +-- service/        # Business logic (7 services)
+-- ai-service/             # Flask AI microservice
|   +-- app.py              # Flask app with 5 endpoints
|   +-- services/           # AI modules
|       | +-- demand_forecast.py # Moving average + ML model foreca
|   sting
|       | +-- stock_health.py    # Stock health scoring
|       | +-- anomaly_detection.py # Z-score anomaly detection
|       | +-- reorder_engine.py  # Automated reorder suggestions
|   +-- models/             # Trained ML models (.pkl)
|   +-- data/               # CSV data files
|   +-- train_model.py      # Model training script
+-- frontend/              # Static frontend (served via XAMP
P)
|   +-- *.html              # 11 HTML pages
|   +-- css/               # style.css, auth.css, dashboard.c
|   ss, responsive.css
|   +-- js/                # 9 JavaScript modules
+-- docs/                  # Project documentation
```

Prerequisites

- Java 21 (JDK)
- Maven 3.9.6 (or use included mvnw)
- XAMPP (MySQL on port 3306)
- Python 3.14+ with pip
- Modern web browser (Chrome, Firefox, Edge)

Setup Instructions

1. Database Setup

Start MySQL via XAMPP Control Panel, then run:

```
CREATE DATABASE inventory_db;  
CREATE USER 'inventory_user'@'localhost' IDENTIFIED BY 'inventory_pass';  
GRANT ALL PRIVILEGES ON inventory_db.* TO 'inventory_user'@'localhost';  
FLUSH PRIVILEGES;
```

2. Start Backend (Spring Boot)

```
cd backend  
mvn spring-boot:run  
# or on Windows:  
.\mvnw.cmd spring-boot:run
```

Backend runs at <http://localhost:8080>. Tables are auto-created by Hibernate.

3. Start AI Service (Flask)

```
cd ai-service  
pip install -r requirements.txt  
python app.py
```

AI service runs at <http://localhost:5000>.

4. Start Frontend

Copy or sync the frontend/ folder to XAMPP's htdocs:

```
C:\xampp\htdocs\inventory\
```

Open <http://localhost/inventory/login.html> in your browser.

5. Login

- Username: admin
- Password: admin123

Features

Core Modules

- Dashboard - KPIs, charts (sales trend, category distribution, top products), recent activity
- Products - CRUD, search, SKU lookup, low-stock alerts, toggle active/inactive, category filtering
- Inventory - Stock IN/OUT/ADJUST movements, current stock calculation, date-range filtering
- Suppliers - CRUD, contact management, email/phone filtering
- Sales - Record sales, auto-calculate totals, status tracking (PENDING/COMPLETED/CANCELLED)
- Purchases - Create purchase orders, receive/cancel workflow, supplier linking
- Reports - Aggregated analytics with Chart.js visualizations
- Profile - User profile viewing

AI-Powered Features

- Demand Forecasting - 7-day product-level demand prediction (moving average + ML model)

- Stock Health Analysis - Per-product health scoring (0-100) with status (healthy/warning/critical)
- Anomaly Detection - Z-score based detection for sales quantity, price, and velocity anomalies
- Reorder Suggestions - Automated reorder quantity calculation with urgency classification
- Natural Language Query - Query the AI system using conversational English

Technical Features

- JWT-based authentication (HS256, 24h expiry)
- Session-based auth (browser close = logout)
- CORS enabled for all origins
- Global exception handling with structured error responses
- Input validation (Jakarta Bean Validation)
- Collapsible filter panels on all data pages
- Responsive design (sidebar, mobile-friendly)
- Bootstrap modal system with custom CSS override

API Endpoints

Auth

Method	Endpoint	Description
POST	<code>`/api/auth/login`</code>	Login, returns JWT
POST	<code>`/api/auth/register`</code>	Register new user

Products (8 endpoints)

Method	Endpoint	Description
GET	<code>`/api/products`</code>	List all products
GET	<code>`/api/products/{id}`</code>	Get product by ID
POST	<code>`/api/products`</code>	Create product
PUT	<code>`/api/products/{id}`</code>	Update product
DELETE	<code>`/api/products/{id}`</code>	Delete product
GET	<code>`/api/products/search?q=`</code>	Search products
GET	<code>`/api/products/low-stock`</code>	Low stock products
PATCH	<code>`/api/products/{id}/toggle-status`</code>	Toggle active/inactive

Suppliers, Inventory, Sales, Purchases, Dashboard

See docs/API_Documentation.pdf for full API reference.

AI Service (Flask - port 5000)

Method	Endpoint	Description
GET	<code>`/api/ai/forecast/{product_id}`</code>	7-day demand forecast
GET	<code>`/api/ai/stock-health`</code>	Stock health analysis
GET	<code>`/api/ai/anomalies`</code>	Anomaly detection
GET	<code>`/api/ai/reorder-suggestions`</code>	Reorder suggestions
POST	<code>`/api/ai/query`</code>	Natural language query

Database Schema

6 tables managed by Hibernate auto-DDL:

```
users      (id, username, email, password, full_name, role, created_at, updated_at)
products   (id, name, description, sku, category, unit_price, cost_p
```

```
rice, reorder_level, image_url, is_active, created_at, updated_at)
suppliers      (id, name, contact_person, email, phone, address, created
_at, updated_at)
sales          (id, product_id FK, quantity, unit_price, total_price, cu
stomer_name, sale_date, status, created_at, updated_at)
purchases      (id, product_id FK, supplier_id FK, quantity, unit_cost,
total_cost, purchase_date, status, created_at, updated_at)
inventory_movements (id, product_id FK, movement_type, quantity, referen
ce_number, notes, created_by, movement_date)
```

Documentation

Document	Description
`docs/Project_Report.md`	Full project report with architecture, code snippe
`docs/Requirements.md`	Functional and non-functional requirements
`docs/Project_Explanation.md`	Module-by-module walkthrough with code
`docs/API_Documentation.pdf`	REST API reference
`docs/Database_Design.pdf`	ER diagram and schema design
`docs/Viva_Questions.pdf`	55 viva preparation Q&As